

INFO0009-2 : Base de données - Projet partie 2

Joachim Amandine, Malaise Romain, Renard Baptiste

5 juillet 2025

1 Description de l'architecture du site Web

Tout d'abord, notre site Web est constitué simplement d'une page principale avec des redirections vers les divers formulaires demandés. Ces formulaires permettent de sélectionner, supprimer, d'insérer ou bien modifier les données relatives à notre base de données. Pour cela, nous avons utilisé l'HTML et le PHP (ainsi que du SQL pour l'initialisation des données). Donc nous avons une interface construite en HTML/PHP et une transmission des requêtes vers SQL via PHP. En effet, PHP reçoit les données des formulaires, les traite, effectue des vérifications sur les contraintes d'intégrité (bien qu'elles soient implémentées en SQL aussi) et renvoie des retours à l'utilisateur (succès, erreur, ...). Nous avons décidé de faire en sorte que PHP vérifie aussi la plupart des contraintes d'intégrité pour pouvoir faire un retour à l'utilisateur plus fluide et plus intuitif au lieu d'obtenir simplement une erreur SQL en retour.

En ce qui concerne l'agencement des pages, nous avons 7 pages qui correspondent aux 7 pages demandées. Elles sont dans l'ordre de l'énoncé le nom du lien sur la page principale donne leur ligne dans le tableau de l'énoncé.

- Partie 2 : Dans cette page, nous avons un tableau comprenant 3 colonnes agence, horaire, exception chaque colonne permet de rechercher dans leur table respective les tuples respectant les contraintes insérées dans les champs de texte. Pour les chaînes de caractère, les contraintes sont de contenance. Pour le reste, elles sont d'égalité (tout ce qui concerne cette page se trouve dans le fichier 1-filtre_agence_horaire_exception.php).
- Partie 3 : Dans cette page, on trouve un formulaire qui sert à ajouter un service dans notre base de données. On doit y mettre un nom, les jours de la semaine concernés, la date de début et la date de fin. On peut aussi y mettre des exceptions sur les jours à ajouter ou retirer dans le service qui ne font pas parties des jours cochés (tout ce qui concerne cette page se trouve dans le fichier 2-ajouter_service.php).
- Partie 4 : Cette page est très simple, on y trouve seulement de quoi rechercher les services disponibles pour une date saisie. Si la date saisie se trouve dans un service alors celui-ci sera affiché, excepté si la date est au delà de 1000 jours (tout ce qui concerne cette page se trouve dans le fichier 3-rechercher_service.php).
- Partie 5 : Sur cette page, on y voit les moyennes des temps d'arrêt classés par ordre alphabétique et par ordre de temps (tout ce qui concerne cette page se trouve dans le fichier 4-temps_moyen.php).
- Partie 6 : Cette page possède deux champs qui permettent de mettre des conditions pour trouver une gare. La première case permet de mettre une contrainte de contenance sur le nom de la gare et la deuxième case permet de mettre une condition sur le nombre minimum d'arrêts pour une gare. Il en ressort une liste de gare avec le service qu'il suit, le nombre d'arrêts, nombre d'arrivées et de départs (tout ce qui concerne cette page se trouve dans le fichier 5-rechercher_gare.php).
- Partie 7 : Dans cette page, on a deux possibilités : supprimer un itinéraire et tous ses trajets à l'aide d'une liste déroulante. On peut aussi ajouter un trajet en commençant par choisir un itinéraire grâce à une liste déroulante auquel on va ajouter le trajet. Après cela, une suite de champs apparaît il faut y mettre toutes les infos nécessaires concernant un trajet (tout ce qui concerne cette page se trouve dans le fichier 6-itineraireSuppr_nouveauTrajet.php).
- Partie 8 : Cette page sert à modifier un arrêt existant. Il y a 6 zones de texte, 2 d'entre elles servent à rechercher l'arrêt que l'on souhaite modifier soit via son ID ou via son nom ou une partie de celui-ci, les 4 restantes servent à mettre les nouvelles données pour l'arrêt (Tout ce qui concerne cette page se trouve dans le fichier 7-modifier_arret.php).

2 Initialisation de la base de données

Tout d'abord, nous avons commencé par définir les attributs avec leur type, que nous avons besoin dans les différentes tables, puis nous avons rajouté les différentes clés primaires et étrangères ainsi que les contraintes d'intégrité. Nous utilisons les paramètres suivants sur chaque table : "ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COLLATE = utf8mb4_0900_ai_ci". Ceci permet que toutes les chaînes de caractère soient encodées en Unicode. Nous utilisons également le moteur InnoDB, comme cela toutes les données sont validées ensemble lors d'un COMMIT ou alors aucune validation n'a lieu (ROLLBACK). Aussi, nous avons veillé à bien mettre la création des tables dans le bon ordre, de telle sorte à ce que nous n'ayons pas de problème avec les clés étrangères. En effet, lorsque l'on définit une clé étrangère, la table à laquelle on fait référence doit déjà être créée. De plus, nous préférons définir sur les clés étrangères ON UPDATE CASCADE ON DELETE CASCADE pour garder une cohérence entre nos données et de ne pas avoir des valeurs nulles pour certains attributs qui sont des clés étrangères. Nous pourrions aussi définir la valeur d'une telle clé étrangère à NULL au lieu de supprimer sa table mais nous ne voyons pas l'intérêt pour ce projet et de plus, toutes les clés primaires référencées par les clés étrangères ne peuvent pas être nulls. Mais nous pouvons remarquer que cela peut être intéressant si l'on souhaite conserver les données.

Pour finir, nous avons des TRIGGERS qui permettent par exemple, de vérifier à l'insertion et à la modification que la date d'une exception est bien comprise dans la date de début et de fin du service auquel il fait référence. Ou bien, vérifier que les coordonnées géographiques se situent bien en Belgique. Aussi, nous avons décidé de faire des TRIGGERS à l'insertion et l'update pour vérifier qu'une chaîne vide n'est pas insérée dans un attribut qui ne prend pas de valeur NULL. Nous avons également une procédure pour une vérification plus complexe de contraintes d'intégrité et quelques vues. La procédure ajouter_horaire permet de faire une vérification avancée lors de l'ajout d'un horaire. Elle prend le numéro de séquence de l'arrêt dont on ajoute l'horaire s'il y en a un. S'il n'y en a pas elle lèvera une erreur, car cela signifie que l'arrêt n'est pas desservi par cet itinéraire et ne fait pas partie de l'itinéraire. Ensuite, elle prend le numéro de séquence plus un ou moins un selon la direction du trajet et prend l'arrêt correspondant à ce numéro de séquence. Après, elle regarde si les heures de cet arrêt pour cet itinéraire et ce trajet en tenant compte de la direction correspondent avec le nouvel horaire que l'on veut ajouter. Nous avons également des vues qui se chargent lors de l'initialisation de base de données mais nous les traiterons dans la question suivante. Remarque : nous avons décidé de définir les clés primaires et étrangères à la fin des tables parce que ça nous semblait propre de toutes les définir au même endroit. Aussi, nous aurions pu appeler notre procédure ajouter_horaire dans un TRIGGER sur l'ajout et la modification de horaire. Cependant, nous avons décidé de ne pas le faire, parce que dans un autre contexte applicatif, nous pourrions vouloir ajouter les horaires dans le désordre ou en insérer au milieu, sans devoir tout reconstruire dans l'ordre.

3 Description des requêtes pour répondre aux questions

3.1 Partie 2 : recherche sur les agences, horaires et exceptions

En ce qui concerne les requêtes qui sont exécutées depuis cette page, elles ne sont pas encadrées par transactions. En effet, cela n'est pas nécessaire car elles ne se limitent qu'à des requêtes de sélection qui ne modifient pas notre base de données. Ces requêtes sont des requêtes construites dynamiquement en PHP. Nous avons besoin de la construire dynamiquement car l'utilisateur n'est pas obligé de remplir tous les champs, il faut donc adapter la requête dynamiquement en fonction de ses choix. Nous avons deux tableaux, un qui permet de faire l'association des variables de la requête avec les données entrées par l'utilisateur pour ensuite le transmettre à PDO pour sécuriser l'exécution via des requêtes préparées. Et l'autre tableau dont tous les éléments sont des parties de la condition WHERE de la requête. Pour les construire, nous faisons simplement des conditions *if* pour vérifier si ce tel champ a été rempli ou non. Une fois ce dernier construit, nous n'avons plus qu'à le concaténer à la requête SQL préalablement défini et en unissant ses éléments par AND. Par exemple, si nous imaginons que l'utilisateur recherche toutes les agences d'Allemagne, nous obtenons finalement une requête comme

```
SELECT * FROM agence WHERE URL LIKE "%.de%" AND SIEGE LIKE "%Germany%";
```

Et cette requête nous donnera très probablement toutes les agences enregistrées qui se trouvent en Allemagne.

3.2 Partie 3 : ajout d'un service avec ses exceptions

Pour cette page, il faut faire attention à gérer correctement les transactions. En effet, nous faisons des insertions de nouveaux tuples qui peuvent mal se passer. Pour ce faire, nous commençons une transaction sur tout le processus d'insertion des tuples. Si nous n'avions pas fait cela mais plutôt faire des transactions séparées sur les exceptions et sur les services, nous aurions eu des problèmes. Par exemple, la première exception aurait été insérée et commit. Et ensuite, si la deuxième n'aurait pas été valide, nous aurions ajouté et terminé la première transaction, insérant la première exception et amenant à une incohérence dans les données. Comme nous avons fait, le service est enregistré avec toutes ses exceptions ou bien rien n'est inséré et nous faisons un rollback, s'il y a une erreur. De plus, les contraintes d'intégrité sur les dates du service, et sur les exceptions sont gérées en SQL (TRIGGER et CHECK) mais aussi en PHP dans la couche applicative. Comme cela nous assurons une cohérence dans les données à tout prix et nous renvoyons une réponse élégante à l'utilisateur. Pour finir, certaine contrainte de formatage dans la zone de texte sont aussi vérifiées comme le fait que chaque ligne soit présentée de la façon suivante DATE CODE_EXCEPTION (exemple 2025-04-22 EXCLUS). Exemple de requête obtenue pour cette exception :

```
INSERT INTO exception(SERVICE_ID, DATE, CODE) VALUES (1, 2025-04-22, 2)
```

3.3 Partie 4 : recherche d'un service pour une date donnée

Pour trouver toutes les dates où un service est actif, nous avons du créer deux vues. La première vue est `dates_période` qui calcule toutes les dates d'un service entre la date de début et la date de fin. Celle-ci comprend deux arguments : `SERVICE_ID` et `date_comprise`. Chaque tuple de la table signifie que la date '`date_comprise`' est comprise entre la première et la dernière date du service dont l'identifiant est '`SERVICE_ID`'.

```

1 CREATE VIEW dates_periode AS
2 WITH RECURSIVE dates_r(SERVICE_ID, date_comprise) AS
3 (
4     SELECT ID as SERVICE_ID, DATE_DEBUT AS date_comprise
5     FROM service
6
7     UNION ALL
8
9     SELECT r.SERVICE_ID, DATE_ADD(r.date_comprise, INTERVAL 1 DAY)
10    FROM dates_r r JOIN service s ON s.ID = r.SERVICE_ID
11    WHERE DATE_ADD(r.date_comprise, INTERVAL 1 DAY) <= s.DATE_FIN AND
12          DATE_ADD(r.date_comprise, INTERVAL 1 DAY) < DATE_ADD(s.DATE_DEBUT,
13          INTERVAL 1000 DAY)
12 )
13 SELECT SERVICE_ID, date_comprise FROM dates_r;

```

Dans la requête non récursive nous obtenons pour chaque service leur date de début comme date comprise. Ensuite, dans la requête récursive, nous voulons ajouter le jour suivant mais à deux conditions. Dans un premier temps, nous vérifions que le jour que nous voulons ajouter n'est pas plus grand que le dernier jour du service. Dans un second temps nous vérifions que la date que nous voulons ajouter n'est pas plus de mille jour après le premier jour, ceci nous permet de respecter la limite d'appels récursifs.

La deuxième vue est dates_actives qui elle retourne toutes les dates actives d'un service.

```

1 CREATE VIEW dates_actives AS
2 SELECT * FROM
3 (
4     SELECT dp.SERVICE_ID, dp.date_comprise
5     FROM dates_periode dp JOIN service s ON dp.SERVICE_ID = s.ID
6     WHERE (DAYOFWEEK(dp.date_comprise) = 2 AND s.LUNDI = 1)
7           OR (DAYOFWEEK(dp.date_comprise) = 3 AND s.MARDI = 1)
8           OR (DAYOFWEEK(dp.date_comprise) = 4 AND s.MERCREDI = 1)
9           OR (DAYOFWEEK(dp.date_comprise) = 5 AND s.JEUDI = 1)
10          OR (DAYOFWEEK(dp.date_comprise) = 6 AND s.VENDREDI = 1)
11          OR (DAYOFWEEK(dp.date_comprise) = 7 AND s.SAMEDI = 1)
12          OR (DAYOFWEEK(dp.date_comprise) = 1 AND s.DIMANCHE = 1)
13
14     UNION
15
16     SELECT SERVICE_ID, DATE AS date_comprise
17     FROM exception
18     WHERE CODE = 1
19
20     EXCEPT
21
22     SELECT SERVICE_ID, DATE AS date_comprise
23     FROM exception
24     WHERE CODE = 2
25 )
26 AS finale;

```

Tout d'abord nous prenons depuis la première vue récursive toutes les dates d'un service compris entre son début et sa fin mais nous n'ajoutons que les dates dont leur jour de la semaine fait partie des jours sélectionnés par le service correspondant. Ensuite nous ajoutons l'ensemble des

jours inclus du service et finalement nous retirons les jours exclus du service.

Pour la page de recherche, il suffit d'utiliser la fonction GROUP_CONCAT pour associer à chaque date l'ensemble des services disponibles.

3.4 Partie 5 : Moyenne de temps d'arrêt par trajet et par itinéraire

Pour cette question, la requête n'est pas compliquée, elle consiste à sélectionner le nom de l'itinéraire, l'id du trajet et de faire la moyenne de la différence de temps entre l'heure d'arrivée et de départ à chaque arrêt pour chaque trajet. On ne prend pas en compte les arrêts de départ et d'arrivée car ceux-ci n'ont pas de temps d'arrêt défini. Nous utilisons WITH ROLLUP pour obtenir les sous totaux pour un itinéraire et aussi pour obtenir le total. En ce qui concerne l'ordre on utilise ORDER BY nom, les temps moyens. La requête finale est celle-ci :

```
1      SELECT itineraire.nom, horaire.trajet_id, SEC_TO_TIME(AVG(
2          TIME_TO_SEC(TIMEDIFF(horaire.HEURE_DEPART, horaire.
3              HEURE_ARRIVEE)))) AS AVG_STOP_TIME FROM horaire
4      JOIN itineraire ON horaire.itineraire_ID = itineraire.ID
      WHERE horaire.HEURE_ARRIVEE IS NOT NULL AND horaire.
          HEURE_DEPART IS NOT NULL
      GROUP BY itineraire.NOM, horaire.TRAJET_ID WITH ROLLUP ORDER BY
          itineraire.NOM, AVG_STOP_TIME;
```

Le problème de cette requête est que le total se met en premier et que les sous totaux ne sont pas forcément après les moyennes de trajet. On résout ce problème en parcourant tous les tuples et en regardant si le nom est null ou si le trajet_id est null. Pour cette requête pas de transactions, même explication que pour la partie 2.

3.5 Partie 6 : Recherche des gares contenant une chaîne donnée avec leurs arrêts par service

Pour cette page, nous avons dû faire une requête un peu plus complexe, donc nous avons décidé de la découper avec une vue. Cette vue est nb_trains_gare_service, elle permet d'obtenir une table avec comme attributs : NOM_GARE, NOM_SERVICE, NB_ARRETS, NB_ARRIVEES, NB_DEPARTS. Cette table donne donc pour chaque gare le nombre de trains qui s'arrêtent, arrivent et partent par service. Ensuite, après avoir reçu une partie ou le nom total d'une gare via le formulaire, nous pouvons lancer la requête suivante :

```
1  SELECT * FROM nb_trains_gare_service
2  WHERE NOM_GARE COLLATE utf8mb4_general_ci LIKE :nom AND (NB_ARRETS >=
3      10 OR NB_ARRIVEES >= 10 OR NB_DEPARTS >= 10)
4
5  ORDER BY NB_ARRETS DESC, NB_ARRIVEES DESC, NB_DEPARTS DESC;
6
7  CREATE VIEW nb_trains_gare_service AS
8  SELECT a.NOM AS NOM_GARE, s.NOM AS NOM_SERVICE, COUNT(*) AS NB_ARRETS,
9      COUNT(h.HEURE_ARRIVEE) AS NB_ARRIVEES, COUNT(h.HEURE_DEPART) AS
10     NB_DEPARTS FROM arret a
    JOIN horaire h ON h.ARRET_ID = a.ID
    JOIN trajet t ON h.TRAJET_ID = t.TRAJET_ID
    JOIN service s ON t.SERVICE_ID = s.ID
    GROUP BY a.ID, s.ID ORDER BY NB_ARRETS DESC, NB_ARRIVEES DESC,
        NB_DEPARTS DESC;
```

Listing 1 – Requête pour filtrer les gares actives contenant "guillemins" et ayant un nombre d'arrêts, d'arrivées ou de départs au moins égal à 10.

A noter que la requête n'est pas sensible à la casse, en effet, nous rajoutons la clause `COLLATE utf8mb4_general_ci` directement dans la requête pour permettre de comparer les chaînes de caractère de manière insensible à la casse. De plus, nous rajoutons quand même une couche supplémentaire en mettant les noms de gares de la requête en minuscule avec `LOWER` et en mettant en minuscule le texte saisi par l'utilisateur (en PHP). Si l'utilisateur ne rentre pas de nombre minimum d'arrêt, ce nombre est défini à 0 par défaut, ne limitant pas la requête.

3.6 Partie 7 : Supprimer un itinéraire, créer un trajet avec ses horaires

Pour cette page, nous avons le premier formulaire qui permet de supprimer un itinéraire, pour celui-ci, nous avons dû faire une requête ainsi que gérer une transaction. Simplement, l'utilisateur choisit l'itinéraire qu'il désire supprimer et une requête sera faite. Par exemple, si l'utilisateur veut supprimer l'itinéraire Eupen — Ostende d'ID 658, la requête sera la suivante :

```
DELETE FROM itineraire WHERE ID = 658
```

Et cette requête est encadrée par une transaction, ce qui permet sa bonne exécution. Si la transaction est validée, l'itinéraire sera supprimé, ainsi que tous ses trajets correspondants. Rappelons nous que nous avons défini sur la table `arret_desservi` la clé étrangère `ITINERAIRE_ID`. Donc, tous les arrêts desservis relatifs à cet itinéraire seront supprimés (parce que nous avons défini la contrainte `ON DELETE CASCADE` dessus). Ce qui se répercutera aussi sur les horaires vu que les arrêts desservis seront supprimés et qu'ils sont définis en tant que clé étrangère sur la table `horaire`. Ainsi, nos données seront dans un état cohérent.

Ensuite, pour le deuxième formulaire, nous avons une nouvelle transaction qui permet la bonne exécution des requêtes et permet une cohérence des données toujours validée. Elle reprend la requête de création du trajet ainsi que la création des horaires. Comme pour la partie 3, nous avons une zone de texte pour saisir les horaires et des contraintes sont appliquées sur ce que l'utilisateur saisi. Par exemple, la première ligne de la zone de texte, étant le premier arrêt de l'horaire, doit avoir seulement une heure de départ, idem pour le dernier arrêt de l'horaire. De plus, chaque arrêt du trajet doit être inclus dans son itinéraire associé. Aussi, chaque arrêt doit être cohérent avec sa séquence dans l'itinéraire. Si un arrêt `x` dans la zone de texte est défini comme desservi, l'arrêt `x+1` a un numéro de séquence plus ou moins grand en fonction de la direction. On s'assure que la séquence est bien croissante ou décroissante. Et pour finir, la séquence des horaires doit être croissante. Par exemple, on ne peut pas avoir (voir le formatage énoncé) :

1	8895000,10:06:00
2	8895067,10:02:00,10:04:00
3	8892007,11:00:00

Ces contraintes sont encore une fois bien sûr gérées dans le SQL via des triggers et procédures mais aussi en PHP avant d'exécuter la requête pour le confort utilisateur. Cependant, le fait que l'utilisateur formate mal la zone de texte fait partie de la couche applicative et est traité dans celle-ci. Cela nous amène aux requêtes suivantes par exemple :

1	INSERT INTO trajet(TRAJET_ID, SERVICE_ID, ITINERAIRE_ID, DIRECTION)
2	VALUES (1, 100, 33, 0)
3	CALL ajouter_horaire(1, 100, 12, 8:00:00, 8:04:00)

La deuxième étant un `CALL` pour appeler la procédure `ajouter_horaire`, qui vérifie une contrainte d'intégrité plus complexe, expliquée dans la partie sur l'initialisation de la base de données.

3.7 Partie 8 :

Dans la page nous avons un formulaire avec 6 champs. Les deux premiers permettent de sélectionner l'arrêt par l'identifiant ou le nom (non exclusif). L'utilisateur doit remplir au moins un de ces deux champs. Les 4 autres champs permettent à l'utilisateur de faire ses modifications : changer l'identifiant, le nom, la longitude et la latitude. Dans le code PHP qui suit nous lançons une transaction avant tout. Dans la première partie, nous nous assurons que l'utilisateur ait rempli au moins un des deux champs de recherche. Si la recherche n'aboutit pas à exactement un résultat, alors nous le signalons à l'utilisateur car il faut exactement un arrêt à modifier et nous ne permettons pas de changer plusieurs arrêts en même temps. En cas d'échec nous effectuons un rollback. Une fois que nous avons vérifié ceci, nous pouvons passer à la deuxième partie : effectuer les modifications de l'utilisateur. Nous n'avons que la position à vérifier, celle-ci doit être en Belgique (nous simulons simplement le bounding box). Si cette vérification échoue alors nous effectuons un rollback et aucune modification de nos données sera effectuée. Cette vérification est effectuée bien entendu en SQL via un trigger et en PHP dans la couche applicative. Finalement nous pouvons envoyer une requête et si sa valeur de retour n'est pas null alors nous faisons le commit et dans le cas inverse, un rollback. Garder toutes ces instructions dans une transaction est très important car nous voulons garder la cohérence entre la première et la deuxième partie.

4 Brève description des distributions de rôles/tâches

Pour ce projet, nous avons utilisé un GIT afin de faciliter nos échanges de code. Pour commencer, chacun de nous s'est vu attribuer une partie de l'énoncé, celui-ci étant déjà divisé en des parties distinctes cela n'a pas été compliqué. Lorsque l'un de nous finissait une partie, ceux du groupe qui n'avaient pas écrit le code vérifiaient si le code paraissait correcte, s'il y avait des bugs ou des possibilités de rendre les choses plus compréhensibles. Après cela, la personne continuait avec la question suivante de l'énoncé qui n'était pas encore attribuée. Grâce à cela le travail a donc été réparti plus ou moins équitablement de plus nous avons pu éviter de bêtes erreurs en nous relisant les uns les autres.